

ICS 604

Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Reminder: Homework Assignment 1

- Due: **11:59 PM** on **Wednesday, February 11**
- Please carefully read and follow the assignment instructions and problem requirements.
- Submit only the Jupyter Notebook file (.ipynb) to Lamaku.
 - Do not upload the data file.
 - Make sure to **rename your notebook file** as instructed.

Lecture 7

- Data Wrangling
 - Groupby and Split-Apply-Combine (Cont'd.)
 - Thinning groups
 - Hierarchical Indexing
 - Joining, Combining and Reshaping Data
 - This section will be left for self-study, as it primarily involves hands-on code execution.

Previously... Split-Apply-Combine paradigm

- `groupby()` is often applied in the context of the data processing paradigm called “split-apply-combine”.
 - **Split:** You need to split the data into chunks defined using one or more columns.
 - This is typically done using `groupby()`.
 - **Apply:** Apply some operation to the chunks generated.
 - Ex. Count the number of rows in each chunk, average the values for a specific column, etc.
 - **Combine:** Combine the results of the applied operation into a new DataFrame.

Thinning groups

- Thinning the data consists in reducing the number of entries in a group.
- As opposed to aggregating functions, thinning does not have to reduce the group into a single entry.
 - Although it could reduce it to a single entry.
- Thinning can be used, for instance, to return only the top 3 entries in each category, or to randomly sample a small subset of entries from each category.

Sub-sampling a DataFrame

- Subsampling is the process of selecting a subset of the original data.
- This can be achieved using the DataFrame method called `sample()`.
 - Two parameters are relevant in this scenario, `n` the number of samples to randomly select or `frac` a portion of the data to return.

```
spending_df.sample(frac=0.001)
```

- Subsampling is an ideal way to handle datasets that are unbalanced across some factor(s).

```
# return 1% of the data, i.e., 100 entries
spending_df.sample(frac=0.01)
```

	doctor_id	specialty	medication	nb_beneficiaries	spending	spending_pct
unique_id						
UH807235	1447230941	INTERNAL MEDICINE	ALENDRONATE SODIUM	24	177.18	0.001820
YS953688	1316947484	INTERNAL MEDICINE	CARISOPRODOL	16	663.66	0.006819
DL782194	1598049322	PHYSICIAN ASSISTANT	METOPROLOL TARTRATE	13	79.09	0.006630
NR674778	1407175995	RHEUMATOLOGY	ADALIMUMAB	13	67835.14	7.232658
DK368407	1821079682	FAMILY PRACTICE	METHYLPREDNISOLONE	42	630.56	0.009677
...	...	...	...	...	...	...
PT710704	1023018199	INTERNAL MEDICINE	FINASTERIDE	89	1914.68	0.019672
KZ114129	1740211952	CARDIOLOGY	NADOLOL	12	1707.47	0.089126
HG169995	1417927500	CARDIOLOGY	DIGOXIN	40	926.40	0.048356
TU699302	1205139110	NURSE PRACTITIONER	FELODIPINE	26	738.02	0.022101
GF819597	1508993882	INTERNAL MEDICINE	EZETIMIBE	29	13277.06	0.136415

100 rows × 6 columns

```
spending_df.sample(n=10)
```

	doctor_id	specialty	medication	nb_beneficiaries	spending	spending_pct
unique_id						
OQ100009	1336131606	INTERNAL MEDICINE	TRIMETHOPRIM	13	338.13	0.003474
CR620944	1447305412	NURSE PRACTITIONER	PROCHLORPERAZINE MALEATE	13	173.94	0.005209
AS925249	1801917786	PSYCHIATRY	OLANZAPINE	24	2574.95	0.147274
WF787498	1740549419	EMERGENCY MEDICINE	HYDROCODONE/ACETAMINOPHEN	11	65.72	0.033023
MQ989205	1932204948	FAMILY PRACTICE	CLONAZEPAM	22	236.47	0.003629
KD206211	1811992183	ORTHOPEDIC SURGERY	IBUPROFEN	61	637.72	0.982648
DE165637	1164528105	GENERAL PRACTICE	BECLOMETHASONE DIPROPIONATE	56	8590.64	1.785306
KU856872	1295816684	FAMILY PRACTICE	PHENYTOIN SODIUM EXTENDED	12	551.88	0.008470
RI149272	1104832534	FAMILY PRACTICE	LEVOTHYROXINE SODIUM	67	3788.95	0.058148
LB658428	1588752067	GERIATRIC MEDICINE	ISOSORBIDE DINITRATE	27	1853.74	0.664588

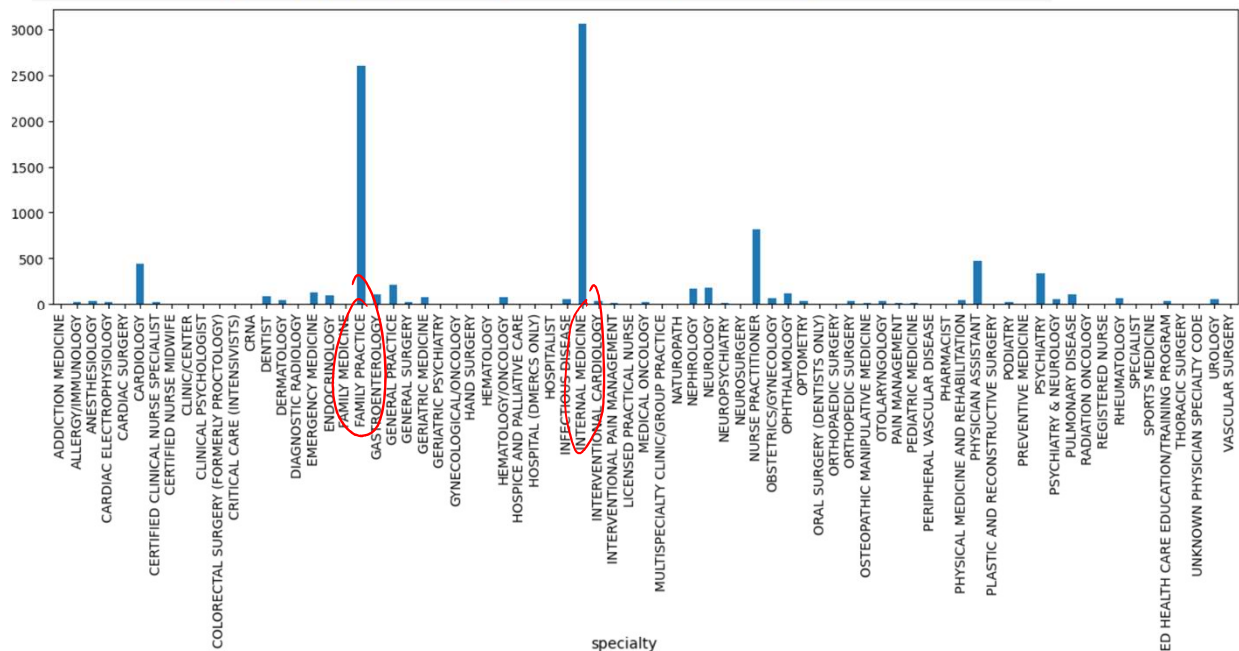
```
spending_df.groupby('specialty').size()
```

```
specialty
ADDICTION MEDICINE      3
ALLERGY/IMMUNOLOGY     24
ANESTHESIOLOGY         30
CARDIAC ELECTROPHYSIOLOGY 20
CARDIAC SURGERY         2
..
STUDENT IN AN ORGANIZED HEALTH CARE EDUCATION/TRAINING PROGRAM 40
THORACIC SURGERY        2
UNKNOWN PHYSICIAN SPECIALTY CODE 4
UROLOGY                 59
VASCULAR SURGERY        2
Length: 75, dtype: int64
```

```
spending_df.groupby('specialty').size().sum()
```

```
10000
```

```
spending_df.groupby('specialty').size().plot.bar(figsize=(16,4))
```



```
# We sample only 10% of the data in each category
```

```
def sample_10p(x):
```

```
    return x.sample(frac=0.1, replace=True)
```

```
spending_by_specialty.apply(sample_10p, include_groups=False).head(10)
```

		doctor_id	medication	nb_beneficiaries	spending	spending_pct
specialty	unique_id					
ALLERGY/IMMUNOLOGY	EW891894	1104888403	DEXLANSOPRAZOLE	32	12411.92	6.561111
	LM829997	1063518546	AMOXICILLIN/POTASSIUM CLAV	20	370.95	0.196089
ANESTHESIOLOGY	WD732008	1538235213	OXYCODONE HCL	56	34073.91	23.860491
	XZ425472	1972502284	METAXALONE	15	4604.98	3.224669
	SC540792	1891850012	HYDROMORPHONE HCL	18	565.06	0.395687
CARDIAC ELECTROPHYSIOLOGY	ZA599500	1538130034	FLECAINIDE ACETATE	69	4270.44	1.893399
	XR445715	1811135908	TRAZODONE HCL	19	69.85	0.030970
CARDIOLOGY	RM548848	1043267958	LOSARTAN/HYDROCHLOROTHIAZIDE	11	219.51	0.011458
	NI413073	1912085705	BISOPROLOL FUMARATE	83	4346.15	0.226860
	IC737150	1003009861	BUMETANIDE	17	340.02	0.017748

```
# We select 10 samples of the data in each category
```

```
def sample_10p(x):
```

```
    return x.sample(n=10, replace=True)
```

```
spending_subsampled_df = spending_df.groupby('specialty').apply(sample_10p, include_groups=False)
```

```
spending_subsampled_df
```

Multi Index

		doctor_id	medication	nb_beneficiaries	spending	spending_pct
specialty	unique_id					
ADDICTION MEDICINE	VG585760	1801032297	LAMOTRIGINE	11	82.62	8.979849
	GJ278932	1134139991	BUSPIRONE HCL	49	817.88	88.894203
	GJ278932	1134139991	BUSPIRONE HCL	49	817.88	88.894203
	TX420809	1801032297	LORAZEPAM	14	19.56	2.125948
	TX420809	1801032297	LORAZEPAM	14	19.56	2.125948
...	...	...	...	...	...	...
VASCULAR SURGERY	KL360002	1710981444	APIXABAN	14	4309.59	98.681300
	CU764903	1780885574	ACETAMINOPHEN WITH CODEINE	16	57.59	1.318700
	CU764903	1780885574	ACETAMINOPHEN WITH CODEINE	16	57.59	1.318700
	KL360002	1710981444	APIXABAN	14	4309.59	98.681300
	CU764903	1780885574	ACETAMINOPHEN WITH CODEINE	16	57.59	1.318700

750 rows x 5 columns

Hierarchical indexing

- We briefly discussed hierarchical indexing in the context of `groupby`.
- Hierarchical indexing allows you to have multiple index levels on an axis.
- Despite the term (indexing), it applies to both indexes and columns (i.e., rows or columns).
- Each index in a `MultiIndex` object is referred to as a level.

```
prog_languages = pd.DataFrame({"Course": ["Python", "Rust", "Python", "Rust"],  
                              "Nb_participants": [10, 30, 27, 18]})  
prog_languages
```

	Course	Nb_participants
0	Python	10
1	Rust	30
2	Python	27
3	Rust	18

```
prog_languages.groupby("Course").sum()
```

	Nb_participants
Course	
Python	37
Rust	48

```
x = prog_languages.groupby("Course")  
p = x.get_group("Python")  
r = x.get_group("Rust")  
  
display(p, r)
```

	Course	Nb_participants
0	Python	10
2	Python	27

	Course	Nb_participants
1	Rust	30
3	Rust	18

```
# MultiIndex created by pairing elements position-wise (zip-style)
data = pd.Series(np.random.randn(9),
                 index=[['a', 'a', 'a', 'b', 'b', 'c', 'c', 'd', 'd'],
                       [1, 2, 3, 1, 3, 1, 2, 2, 3]])
```

data

```
a 1 -1.602250
   2  1.180097
   3 -1.227541
b 1  0.349098
   3  1.082128
c 1 -0.294489
   2 -0.782427
d 2 -0.650776
   3 -1.054067
dtype: float64
```

data.index

```
MultiIndex([(a, 1),
            (a, 2),
            (a, 3),
            (b, 1),
            (b, 3),
            (c, 1),
            (c, 2),
            (d, 2),
            (d, 3)],
           )
```

level 0
level 1

data

```
a 1 -1.602250
   2  1.180097
   3 -1.227541
b 1  0.349098
   3  1.082128
c 1 -0.294489
   2 -0.782427
d 2 -0.650776
   3 -1.054067
dtype: float64
```

```
print(data[('b', 1)])
0.3490980820378435

print(data['b', 1])
0.3490980820378435
```

data[('b',)]

```
1 0.349098
3 1.082128
dtype: float64
```

data['b':'c']

```
b 1 0.349098
   3 1.082128
c 1 -0.294489
   2 -0.782427
dtype: float64
```

data.loc[['b', 'd']]

```
b 1 0.349098
   3 1.082128
d 2 -0.650776
   3 -1.054067
dtype: float64
```

```
# selection from an "inner" level
data.loc[:, 2]
```

```
a 1.180097
c -0.782427
d -0.650776
dtype: float64
```

```
df = pd.DataFrame({"A": np.random.randn(9), "B": np.random.randn(9)},
                  index=[['a', 'a', 'a', 'b', 'b', 'c', 'c', 'd', 'd'], [1, 2, 3, 1, 3, 1, 2, 2, 3]])
df
```

		A	B
a	1	0.464607	0.389820
	2	-0.738748	0.462232
	3	-0.149320	-0.114990
b	1	0.989294	1.488239
	3	1.551830	-0.076878
c	1	1.230385	-0.999217
	2	-0.731376	-0.037591
d	2	-2.041680	0.201706
	3	0.163337	0.112030

*# data.loc[:, 2] worked on a Series but
df.loc[:, 2] does not work on this DataFrame*

```
df.loc[:, 2]
```

KeyError Trace

```
df.loc[df.index.get_level_values(1) == 2]
```

		A	B
a	2	-0.738748	0.462232
c	2	-0.731376	-0.037591
d	2	-2.041680	0.201706

		A	B
a	1	0.464607	0.389820
	2	-0.738748	0.462232
	3	-0.149320	-0.114990
b	1	0.989294	1.488239
	3	1.551830	-0.076878
c	1	1.230385	-0.999217
	2	-0.731376	-0.037591
d	2	-2.041680	0.201706
	3	0.163337	0.112030

#E.g., data.loc[:, 2] worked on a Series but does not work on a DataFrame

```
df.loc(axis=0)[pd.IndexSlice[:, 2]]
```

		A	B
a	2	-0.738748	0.462232
c	2	-0.731376	-0.037591
d	2	-2.041680	0.201706

```
df.loc[pd.IndexSlice[:, 2], :]
```

		A	B
a	2	-0.738748	0.462232
c	2	-0.731376	-0.037591
d	2	-2.041680	0.201706

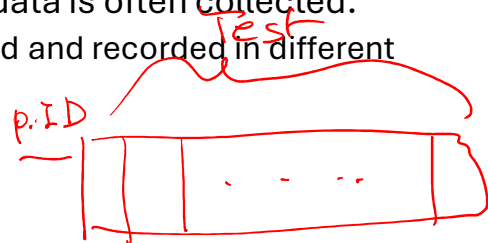
Stacked vs. unstacked data

- Hierarchical indexes represent how the data is often collected.
 - Ex. if variables in a hospital are measured and recorded in different files

File 1

patient_ABC LDL 112
 patient_ABC HDL 48
 patient_CCX LDL 112
 patient_VDM LDL 112
 patient_ABC VO2 112
 patient_CCZ RER 48
 ...

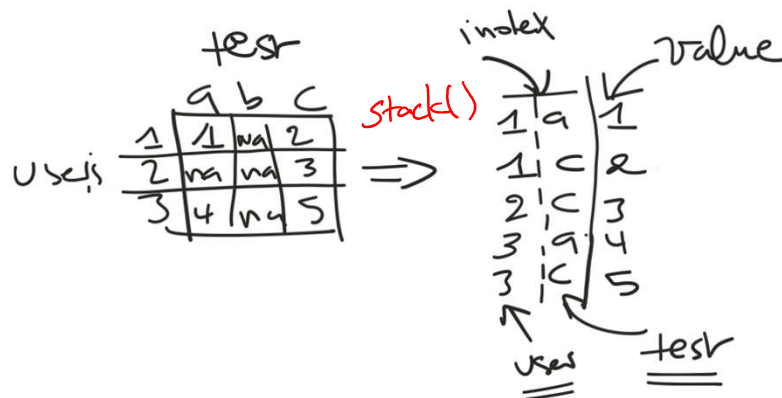
Multi-Index



- This format is preferred when not all the variables are measured in all the patients.
 - I.e., using columns for the variables would result in a large number of empty cells.

Stacking and unstacking a DataFrame

- You can convert back and forth between a DataFrame and a hierarchical representation with `stack()` and `unstack()`.



```
x = pd.DataFrame(np.random.randn(12).reshape((4, 3)),
                 index=list("abcd"), columns=[1, 2, 3])
x.loc["b", 2] = np.nan
x.loc["c", 3] = np.nan
x.loc["d", 1] = np.nan
x
```

	1	2	3
a	-2.413760	-1.201456	0.485889
b	-0.829503	NaN	0.663377
c	0.305271	-0.788319	NaN
d	NaN	-3.101246	0.079588

```
x = x.stack()
```

```
x
a 1 -2.413760
  2 -1.201456
  3  0.485889
b 1 -0.829503
  3  0.663377
c 1  0.305271
  2 -0.788319
d 2 -3.101246
  3  0.079588
dtype: float64
```

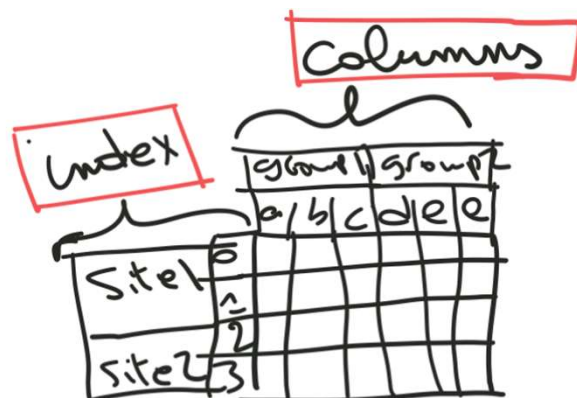
```
x = x.unstack()
```

```
x
```

	1	2	3
a	-2.413760	-1.201456	0.485889
b	-0.829503	NaN	0.663377
c	0.305271	-0.788319	NaN
d	NaN	-3.101246	0.079588

Hierarchical indexes on rows and columns

- In a DataFrame, both axes can have a hierarchical index.
- Always remember the labels are stored under `index` for the rows and `columns` for the columns.



```
data = pd.DataFrame(np.arange(12).reshape((4, 3)),
                    index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]],
                    columns=[['Ohio', 'Ohio', 'Colorado'],
                             ['Green', 'Red', 'Green']])
```

data

		Ohio		Colorado
		Green	Red	Green
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

name hierarchical levels

```
data.index.names = ['key1', 'key2']
data.columns.names = ['state', 'color']
data
```

		state		Ohio	Colorado
		color	Green	Red	Green
key1	key2				
a	1	0	1	2	
	2	3	4	5	
b	1	6	7	8	
	2	9	10	11	

(level=)0

```
print(data.index.get_level_values("key1"))
```

```
Index(['a', 'a', 'b', 'b'], dtype='object', name='key1')
```

```
print(data.index.get_level_values("key2"))
```

```
Index([1, 2, 1, 2], dtype='int64', name='key2')
```

```
print(data.columns.get_level_values("state"))
```

```
Index(['Ohio', 'Ohio', 'Colorado'], dtype='object', name='state')
```

```
print(data.columns.get_level_values("color"))
```

```
Index(['Green', 'Red', 'Green'], dtype='object', name='color')
```

		state		Ohio	Colorado
		color	Green	Red	Green
key1	key2				
a	1	0	1	2	
	2	3	4	5	
b	1	6	7	8	
	2	9	10	11	

Questions

- What type of variable would the following return (DataFrame, Series, np.array, ...)?

`data['Ohio']`

- How about the following?

`data['Ohio', 'Green']`

	state		Ohio	Colorado	
	color		Green	Red	Green
key1	key2				
a	1		0	1	2
	2		3	4	5
b	1		6	7	8
	2		9	10	11

```
print(type(data['Ohio']))
```

```
data['Ohio']
```

```
<class 'pandas.core.frame.DataFrame'>
```

	color	Green	Red
key1	key2		
a	1	0	1
	2	3	4
b	1	6	7
	2	9	10

```
print(type(data['Ohio', 'Green']))
```

```
data['Ohio', 'Green']
```

```
<class 'pandas.core.series.Series'>
```

	key1	key2
a	1	0
	2	3
b	1	6
	2	9

Name: (Ohio, Green), dtype: int64

```
data.loc[:, pd.IndexSlice['Ohio', ['Red', 'Green']]]
```

		state	Ohio	
		color	Red	Green
key1	key2			
a	1		1	0
	2		4	3
b	1		7	6
	2		10	9

		state	Ohio		Colorado
		color	Green	Red	Green
key1	key2				
a	1		0	1	2
	2		3	4	5
b	1		6	7	8
	2		9	10	11

Creating and assigning a MultiIndex object

- You can create a MultiIndex using one of the constructors.
 - Ex. create from arrays
- As long as the dimensions are compatible, you can assign the MultiIndex to either the index or columns attribute of a DataFrame or a Series.

```
data = pd.DataFrame(np.arange(12).reshape((4, 3)),
                    index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]])
data.index.names = ["letter", "number"]
data
```

		0	1	2
letter	number			
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

```
col_idx = pd.MultiIndex.from_arrays([['Ohio', 'Ohio', 'Colorado'], ['Green', 'Red', 'Green']],
                                   names=['state', 'color'])
col_idx
```

```
MultiIndex([(    'Ohio', 'Green'),
            (    'Ohio', 'Red'),
            ('Colorado', 'Green')],
            names=['state', 'color'])
```

```
data.columns = col_idx
data
```

		state		Ohio		Colorado
		color	Green	Red		Green
letter	number					
a	1		0	1		2
	2		3	4		5
b	1		6	7		8
	2		9	10		11

```
row_idx = pd.MultiIndex.from_product(['a', 'b'], [1, 2], names=['letter', 'number'])
row_idx
```

```
MultiIndex([(a, 1),
            (a, 2),
            (b, 1),
            (b, 2)],
           names=['letter', 'number'])
```

Cartesian product : $2 \times 2 = 4$ index values

```
data = pd.DataFrame(np.arange(12).reshape((4, 3)), index=row_idx, columns=col_idx)
data
```

		state		Ohio	Colorado
		color	Green	Red	Green
letter	number				
a	1		0	1	2
	2		3	4	5
b	1		6	7	8
	2		9	10	11

pd.MultiIndex ?

Reordering and sorting levels

- `Swaplevel()` method takes two level numbers or names and returns a new object with the levels interchanged.
- As with simple indexes, you can sort a single or a combination of indexes of the `MultiIndex` object.

```
data = pd.DataFrame(np.arange(12).reshape((4, 3)),
                    index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]])
data.index.names = ['key1', 'key2']
data
```

		0	1	2
key1	key2			
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

```
data.swaplevel('key1', 'key2')
```

		0	1	2
key2	key1			
1	a	0	1	2
2	a	3	4	5
1	b	6	7	8
2	b	9	10	11

```
data.sort_index(key2level=1, ascending=False)
```

sort_remaining = True

		0	1	2
key1	key2			
b	2	9	10	11
a	2	3	4	5
b	1	6	7	8
a	1	0	1	2

```
# You can also pass the level's index number, instead of name
data.swaplevel(0, 1).sort_index(key2level=0, ascending=False)
```

		0	1	2
key2	key1			
2	b	9	10	11
	a	3	4	5
1	b	6	7	8
	a	0	1	2

//
key2

```
data.columns = pd.MultiIndex.from_arrays([[ 'Ohio', 'Ohio', 'Colorado'], [ 'Green', 'Red', 'Green']],
names=[ 'state', 'color'])
```

data

		state		Ohio	Colorado
		color	Green	Red	Green
key1	key2				
a	1		0	1	2
	2		3	4	5
b	1		6	7	8
	2		9	10	11

The 3rd parameter specifies the **axis**; by default, **axis=0**
data.swaplevel(0, 1, 0) $\Leftrightarrow$ swaplevel(0,1)

		state		Ohio	Colorado
		color	Green	Red	Green
key2	key1				
1	a		0	1	2
2	a		3	4	5
1	b		6	7	8
2	b		9	10	11

```
data.swaplevel(0, 1, 1)
```

		color	Green	Red	Green
		state	Ohio	Ohio	Colorado
key1	key2				
a	1		0	1	2
	2		3	4	5
b	1		6	7	8
	2		9	10	11

```
data.swaplevel('state', 'color', axis=1)
```

		color	Green	Red	Green
		state	Ohio	Ohio	Colorado
key1	key2				
a	1		0	1	2
	2		3	4	5
b	1		6	7	8
	2		9	10	11

Summary statistics by level

- The summary statistics that we've seen before can also be applied by level.
- When you apply a function on the level, the remaining levels get “squashed”.
- The group level functions can be obtained using `groupby`.

data					data.groupby(level='key1').sum()					data.groupby(level='key2').sum()				
state		Ohio		Colorado	state		Ohio		Colorado	state		Ohio		Colorado
color	Green	Red	Green		color	Green	Red	Green		color	Green	Red	Green	
key1	key2				key1					key2				
a	1	0	1	2	a	3	5	7		1	6	8	10	
	2	3	4	5										
b	1	6	7	8	b	15	17	19		2	12	14	16	
	2	9	10	11										

```
data.groupby(level="state", axis=1).sum()
```

C:\Users\kayba\AppData\Local\Temp\ipykernel_14520\2207453615.py:1: FutureWarning: DataFrame.groupby with axis=1 is deprecated. Do `frame.T.groupby(...)` without axis instead.

```
data.groupby(level="state", axis=1).sum()
```

	state	Colorado	Ohio
key1	key2		

a	1	2	1
	2	5	7
b	1	8	13
	2	11	19

state color
key1
key2
`data.T.groupby(level='state').sum().T`

	state	Colorado	Ohio
key1	key2		

a	1	2	1
	2	5	7
b	1	8	13
	2	11	19

	state		Ohio	Colorado
	color	Green	Red	Green
key1	key2			
a	1	0	1	2
	2	3	4	5
b	1	6	7	8
	2	9	10	11

Setting and resetting the index

- It's common to read in a DataFrame file and use one or more of its columns as the index.
 - Setting one or more columns as the index is done using `set_index()`.
 - By default, the columns are removed from the DataFrame, but you can choose to leave them in.
 - `reset_index()` does the opposite; moves the index as a new column of the DataFrame.
 - Creates a new index from an `RangeIndex()`.

```
data = pd.DataFrame({'a': range(7), 'b': range(7, 0, -1),
                    'c': ['one', 'one', 'one', 'two', 'two', 'two', 'two'],
                    'd': [0, 1, 2, 0, 1, 2, 3]})
```

data

	a	b	c	d
0	0	7	one	0
1	1	6	one	1
2	2	5	one	2
3	3	4	two	0
4	4	3	two	1
5	5	2	two	2
6	6	1	two	3

```
data2 = data.set_index('b')
data2
```

	a	c	d
b			
7	0	one	0
6	1	one	1
5	2	one	2
4	3	two	0
3	4	two	1
2	5	two	2
1	6	two	3

```
data3 = data.set_index(['c', 'd'])
data3
```

	a	b
c d		
one 0	0	7
1 1	6	
2 2	5	
two 0	3	4
1 4	3	
2 5	2	
3 6	1	

```
data.set_index(['c', 'd'], drop=False)
```

	a	b	c	d
c d				
one 0	0	7	one	0
1 1	6	one	1	
2 2	5	one	2	
two 0	3	4	two	0
1 4	3	two	1	
2 5	2	two	2	
3 6	1	two	3	

```
display(data2)
```

```
data4 = data2.reset_index()  
data4
```

data2

	a	c	d
b			
7	0	one	0
6	1	one	1
5	2	one	2
4	3	two	0
3	4	two	1
2	5	two	2
1	6	two	3

data4

	b	a	c	d
0	7	0	one	0
1	6	1	one	1
2	5	2	one	2
3	4	3	two	0
4	3	4	two	1
5	2	5	two	2
6	1	6	two	3

```
data4.index
```

```
RangeIndex(start=0, stop=7, step=1)
```

```
display(data3)  
data3.reset_index((1,))
```

data3

	a	b	d	a	b		
c	d		c				
one	0	0	7	one	0	0	7
	1	1	6	one	1	1	6
	2	2	5	one	2	2	5
two	0	3	4	two	0	3	4
	1	4	3	two	1	4	3
	2	5	2	two	2	5	2
	3	6	1	two	3	6	1

```
data3.reset_index((1,), drop=True)
```

	a	b
c		
one	0	7
one	1	6
one	2	5
two	3	4
two	4	3
two	5	2
two	6	1

Joining, Combining and Reshaping Data

Combining and merging datasets

- You can combine DataFrames using different strategies.
 - You can achieve similar effect as join in SQL.
 - You can also concatenate DataFrames horizontally or vertically.
- The operation we will cover and use are:
 - `merge()`: connect rows in DataFrames based on keys (similar to database join operation)
 - `concat()`: concatenate data (horizontally) or stacks it (vertically)

```
df1 = pd.DataFrame({'key': ['b', 'b', 'a', 'c', 'a', 'a', 'b'], 'data1': range(7)})
df1
```

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	a	5
6	b	6

```
df2 = pd.DataFrame({'key': ['a', 'b', 'd'], 'data2': range(3)})
df2
```

	key	data2
0	a	0
1	b	1
2	d	2

Database-style DataFrame joins

- Merge on a common keys.
- We can merge df1 and df2 on the common column name key.
 - Pandas can infer the overlapping key, or you can specify it explicitly using on=.
 - Using on= avoids mistakes in large dataset with dozens of columns.
- Since some values are duplicated in df1 while df2 has a single row for a key value, this amount to a many-to-one join.

df1

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	a	5
6	b	6

df2

	key	data2
0	a	0
1	b	1
2	d	2

```
merged_df = pd.merge(df1, df2)  
merged_df
```

	key	data1	data2
0	b	0	1
1	b	1	1
2	a	2	0
3	a	4	0
4	a	5	0
5	b	6	1

df1

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	a	5
6	b	6

df2

	key	data2
0	a	0
1	b	1
2	d	2

```
merged_df = pd.merge(df1, df2, on='key')  
merged_df
```

	key	data1	data2
0	b	0	1
1	b	1	1
2	a	2	0
3	a	4	0
4	a	5	0
5	b	6	1

```
merged_df.sort_values("key")
```

	key	data1	data2
2	a	2	0
3	a	4	0
4	a	5	0
0	b	0	1
1	b	1	1
5	b	6	1

- If the columns are different in both datasets, you can specify them separately using `left_on=` and `right_on=`.

```
df3 = pd.DataFrame({'lkey': ['b', 'b', 'a', 'c', 'a', 'a', 'b'], 'data1': range(7)})
df4 = pd.DataFrame({'rkey': ['a', 'b', 'd'], 'data2': range(3)})
display(df3)
display(df4)
merged_df = pd.merge(df3, df4, left_on='lkey', right_on='rkey')
merged_df[["lkey", "rkey", "data1", "data2"]]
```

	lkey	data1		rkey	data2		lkey	rkey	data1	data2
0	b	0	0	a	0	0	b	b	0	1
1	b	1	1	b	1	1	b	b	1	1
2	a	2	2	d	2	2	a	a	2	0
3	c	3				3	a	a	4	0
4	a	4				4	a	a	5	0
5	a	5				5	b	b	6	1
6	b	6								

Unmatched values

lkey data1			rkey data2		
0	b	0	0	a	0
1	b	1	1	b	1
2	a	2	2	d	2
3	c	3			
4	a	4			
5	a	5			
6	b	6			

- What happened to lkey=c and rkey=d? Why aren't they showing in the resulting DataFrame?
 - Unmatched keys are dropped from the merged result.

Merge types

- By default, the merge does an inner join.
- Different merge types in `pandas.merge()`.

Option	Behavior
inner	Keeps only keys present in both DataFrames (intersection)
left	Keeps all keys from the left DataFrame
right	Keeps all keys from the right DataFrame
outer	Keeps all keys from both DataFrame (union)
cross	Returns the Cartesian product of the two DataFrames (cross join)

df1

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	a	5
6	b	6

df2

	key	data2
0	a	0
1	b	1
2	d	2

```
pd.merge(df1, df2, on='key', how='outer')
```

	key	data1	data2
0	a	2.0	0.0
1	a	4.0	0.0
2	a	5.0	0.0
3	b	0.0	1.0
4	b	1.0	1.0
5	b	6.0	1.0
6	c	3.0	NaN
7	d	NaN	2.0

```
pd.merge(df3, df4, left_on='lkey', right_on='rkey', how='outer')
```

	lkey	data1	rkey	data2
0	a	2.0	a	0.0
1	a	4.0	a	0.0
2	a	5.0	a	0.0
3	b	0.0	b	1.0
4	b	1.0	b	1.0
5	b	6.0	b	1.0
6	c	3.0	NaN	NaN
7	NaN	NaN	d	2.0

df3

	lkey	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	a	5
6	b	6

df4

	rkey	data2
0	a	0
1	b	1
2	d	2

```
df1 = pd.DataFrame({'key': ['b', 'b', 'a', 'c', 'a', 'b'],
                    'data1': range(6)})
```

df1

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	b	5

```
df2 = pd.DataFrame({'key': ['a', 'b', 'a', 'b', 'd'],
                    'data2': range(5)})
```

df2

	key	data2
0	a	0
1	b	1
2	a	2
3	b	3
4	d	4

df1

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	b	5

df2

	key	data2
0	a	0
1	b	1
2	a	2
3	b	3
4	d	4

```
pd.merge(df1, df2, how='inner')
```

	key	data1	data2
0	b	0	1
1	b	0	3
2	b	1	1
3	b	1	3
4	a	2	0
5	a	2	2
6	a	4	0
7	a	4	2
8	b	5	1
9	b	5	3

df1

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	b	5

df2

	key	data2
0	a	0
1	b	1
2	a	2
3	b	3
4	d	4

```
# many-to-many merge
pd.merge(df1, df2, on='key', how='left')
```

	key	data1	data2
0	b	0	1.0
1	b	0	3.0
2	b	1	1.0
3	b	1	3.0
4	a	2	0.0
5	a	2	2.0
6	c	3	NaN
7	a	4	0.0
8	a	4	2.0
9	b	5	1.0
10	b	5	3.0

df1

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	b	5

df2

	key	data2
0	a	0
1	b	1
2	a	2
3	b	3
4	d	4

```
pd.merge(df1, df2, how='right')
```

	key	data1	data2
0	a	2.0	0
1	a	4.0	0
2	b	0.0	1
3	b	1.0	1
4	b	5.0	1
5	a	2.0	2
6	a	4.0	2
7	b	0.0	3
8	b	1.0	3
9	b	5.0	3
10	d	NaN	4

df1

	key	data1
0	b	0
1	b	1
2	a	2
3	c	3
4	a	4
5	b	5

df2

	key	data2
0	a	0
1	b	1
2	a	2
3	b	3
4	d	4

```
pd.merge(df1, df2, how='cross').head(30)
```

	key_x	data1	key_y	data2
0	b	0	a	0
1	b	0	b	1
2	b	0	a	2
3	b	0	b	3
4	b	0	d	4
...	...	...	...	...
25	b	5	a	0
26	b	5	b	1
27	b	5	a	2
28	b	5	b	3
29	b	5	d	4

30 rows × 4 columns

```

left = pd.DataFrame({'key1': ['foo', 'foo', 'bar'],
                      'key2': ['one', 'two', 'one'],
                      'lval': [1, 2, 3]})

left

```

	key1	key2	lval
0	foo	one	1
1	foo	two	2
2	bar	one	3

```

right = pd.DataFrame({'key1': ['foo', 'foo', 'bar', 'bar'],
                      'key2': ['one', 'one', 'one', 'two'],
                      'rval': [4, 5, 6, 7]})

right

```

	key1	key2	rval
0	foo	one	4
1	foo	one	5
2	bar	one	6
3	bar	two	7

left			
	key1	key2	lval
0	foo	one	1
1	foo	two	2
2	bar	one	3

right			
	key1	key2	rval
0	foo	one	4
1	foo	one	5
2	bar	one	6
3	bar	two	7


```
# merge with multiple keys
```

```
pd.merge(left, right, on=['key1', 'key2'], how='outer')
```


	key1	key2	lval	rval
0	bar	one	3.0	6.0
1	bar	two	NaN	7.0
2	foo	one	1.0	4.0
3	foo	one	1.0	5.0
4	foo	two	2.0	NaN

Avoid column collision

- Pandas renames similar columns to avoid ambiguity.
- When `merge()` creates ambiguity, Pandas renames identical columns across tables, adding suffix `_x` to the left frame and `_y` to the right frame by default.

left

	key1	key2	lval
0	foo	one	1
1	foo	two	2
2	bar	one	3

right

	key1	key2	rval
0	foo	one	4
1	foo	one	5
2	bar	one	6
3	bar	two	7

```
pd.merge(left, right, on='key1')
```

	key1	key2_x	lval	key2_y	rval
0	foo	one	1	one	4
1	foo	one	1	one	5
2	foo	two	2	one	4
3	foo	two	2	one	5
4	bar	one	3	one	6
5	bar	one	3	two	7

left

	key1	key2	lval
0	foo	one	1
1	foo	two	2
2	bar	one	3

right

	key1	key2	rval
0	foo	one	4
1	foo	one	5
2	bar	one	6
3	bar	two	7

```
pd.merge(left, right, on='key1', suffixes=('_left', '_right'))
```

	key1	key2_left	lval	key2_right	rval
0	foo	one	1	one	4
1	foo	one	1	one	5
2	foo	two	2	one	4
3	foo	two	2	one	5
4	bar	one	3	one	6
5	bar	one	3	two	7

Merging on index

- Merging on the index is also very common.
- Merge on one or more indexes with hierarchical keys operates the same way as a merge on multiple columns.

```
left1 = pd.DataFrame({'key': ['a', 'b', 'a', 'a', 'b', 'c'],  
                      'value': range(6)})
```

left1

	key	value
0	a	0
1	b	1
2	a	2
3	a	3
4	b	4
5	c	5

```
right1 = pd.DataFrame({'group_val': [3.5, 7]}, index=['a', 'b'])  
right1
```

	group_val
a	3.5
b	7.0

left1						
	key	value				
0	a	0				
1	b	1				
2	a	2				
3	a	3				
4	b	4				
5	c	5				
right1						
	group_val					
a	3.5					
b	7.0					

```
pd.merge(left1, right1, left_on='key', right_index=True)
```

	key	value	group_val
0	a	0	3.5
1	b	1	7.0
2	a	2	3.5
3	a	3	3.5
4	b	4	7.0

left1						
	key	value				
0	a	0				
1	b	1				
2	a	2				
3	a	3				
4	b	4				
5	c	5				
right1						
	group_val					
a	3.5					
b	7.0					

```
pd.merge(left1, right1, left_on='key', right_index=True, how='outer')
```

	key	value	group_val
0	a	0	3.5
2	a	2	3.5
3	a	3	3.5
1	b	1	7.0
4	b	4	7.0
5	c	5	NaN

```
left_hierarchical = pd.DataFrame({'key1': ['Ohio', 'Ohio', 'Ohio', 'Nevada', 'Nevada'],
                                  'key2': [2000, 2001, 2002, 2001, 2002],
                                  'data': np.arange(5.)})
left_hierarchical
```

	key1	key2	data
0	Ohio	2000	0.0
1	Ohio	2001	1.0
2	Ohio	2002	2.0
3	Nevada	2001	3.0
4	Nevada	2002	4.0

```
right_hierarchical = pd.DataFrame(np.arange(12).reshape((6, 2)),
                                  index=[['Nevada', 'Nevada', 'Ohio', 'Ohio', 'Ohio', 'Ohio'],
                                          [2001, 2000, 2000, 2000, 2001, 2002]],
                                  columns=['event1', 'event2'])
right_hierarchical
```

		event1	event2
Nevada	2001	0	1
	2000	2	3
Ohio	2000	4	5
	2000	6	7
	2001	8	9
	2002	10	11

left_hierarchical

	key1	key2	data
0	Ohio	2000	0.0
1	Ohio	2001	1.0
2	Ohio	2002	2.0
3	Nevada	2001	3.0
4	Nevada	2002	4.0

right_hierarchical

		event1	event2
Nevada	2001	0	1
	2000	2	3
Ohio	2000	4	5
	2000	6	7
	2001	8	9
	2002	10	11

```
pd.merge(left_hierarchical, right_hierarchical,  
         left_on=['key1', 'key2'], right_index=True)
```

	key1	key2	data	event1	event2
0	Ohio	2000	0.0	4	5
0	Ohio	2000	0.0	6	7
1	Ohio	2001	1.0	8	9
2	Ohio	2002	2.0	10	11
3	Nevada	2001	3.0	0	1

left_hierarchical

	key1	key2	data
0	Ohio	2000	0.0
1	Ohio	2001	1.0
2	Ohio	2002	2.0
3	Nevada	2001	3.0
4	Nevada	2002	4.0

right_hierarchical

		event1	event2
Nevada	2001	0	1
	2000	2	3
Ohio	2000	4	5
	2000	6	7
	2001	8	9
	2002	10	11

```
pd.merge(left_hierarchical, right_hierarchical,  
         left_on=['key1', 'key2'], right_index=True, how='outer')
```

	key1	key2	data	event1	event2
4	Nevada	2000	NaN	2.0	3.0
3	Nevada	2001	3.0	0.0	1.0
4	Nevada	2002	4.0	NaN	NaN
0	Ohio	2000	0.0	4.0	5.0
0	Ohio	2000	0.0	6.0	7.0
1	Ohio	2001	1.0	8.0	9.0
2	Ohio	2002	2.0	10.0	11.0

```
left2 = pd.DataFrame([[1., 2.], [3., 4.], [5., 6.]],
                     index=['a', 'c', 'e'],
                     columns=['Ohio', 'Nevada'])
```

left2

	Ohio	Nevada
a	1.0	2.0
c	3.0	4.0
e	5.0	6.0

```
right2 = pd.DataFrame([[7., 8.], [9., 10.], [11., 12.], [13., 14.]],
                      index=['b', 'c', 'd', 'e'],
                      columns=['Missouri', 'Alabama'])
```

right2

	Missouri	Alabama
b	7.0	8.0
c	9.0	10.0
d	11.0	12.0
e	13.0	14.0

left2

	Ohio	Nevada
a	1.0	2.0
c	3.0	4.0
e	5.0	6.0

right2

	Missouri	Alabama
b	7.0	8.0
c	9.0	10.0
d	11.0	12.0
e	13.0	14.0

```
pd.merge(left2, right2, how='outer', left_index=True, right_index=True)
```

	Ohio	Nevada	Missouri	Alabama
a	1.0	2.0	NaN	NaN
b	NaN	NaN	7.0	8.0
c	3.0	4.0	9.0	10.0
d	NaN	NaN	11.0	12.0
e	5.0	6.0	13.0	14.0

Concatenating along an axis

- Also known as binding or stacking
- By default, concatenation
 - operates on `axis=0`
 - uses `join=outer`

```
arr = np.arange(12).reshape((3, 4))  
arr
```

```
array([[ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11]])
```

```
np.concatenate([arr, arr])
```

```
array([[ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11],  
       [ 0,  1,  2,  3],  
       [ 4,  5,  6,  7],  
       [ 8,  9, 10, 11]])
```

```
np.concatenate([arr, arr], axis=1)
```

```
array([[ 0,  1,  2,  3,  0,  1,  2,  3],  
       [ 4,  5,  6,  7,  4,  5,  6,  7],  
       [ 8,  9, 10, 11,  8,  9, 10, 11]])
```

```

s1 = pd.Series([0, 1], index=['a', 'b'])
s2 = pd.Series([2, 3, 4], index=['c', 'd', 'e'])
s3 = pd.Series([5, 6], index=['f', 'g'])

print(s1)

print("*" * 10)
print(s2)

print("*" * 10)
print(s3)

```

```

a    0
b    1
dtype: int64
*****
c    2
d    3
e    4
dtype: int64
*****
f    5
g    6
dtype: int64

```

```
pd.concat([s1, s2, s3])
```

```

a    0
b    1
c    2
d    3
e    4
f    5
g    6
dtype: int64

```

```
pd.concat([s1, s2, s3], axis=1)
```

	0	1	2
a	0.0	NaN	NaN
b	1.0	NaN	NaN
c	NaN	2.0	NaN
d	NaN	3.0	NaN
e	NaN	4.0	NaN
f	NaN	NaN	5.0
g	NaN	NaN	6.0

```

print(s1)
print(" *" * 10)
print(s3)

s4 = pd.concat([s1, s3])
s4

a    0
b    1
dtype: int64
*****
f    5
g    6
dtype: int64

a    0
b    1
f    5
g    6
dtype: int64

```

```
pd.concat([s1, s4], axis=1)
```

	0	1
a	0.0	0
b	1.0	1
f	NaN	5
g	NaN	6

```
display(pd.concat([s1, s4], axis=1, join='inner'))
```

	0	1
a	0	0
b	1	1

Creating indexes by concatenation

- Concatenation is useful for creating a hierarchical index of the data.
 - Use the `keys=` parameter, with the same length as the array of objects being concatenated.
 - If `axis=1`, keys become the column (header) names when concatenating Series.
 - If `axis=0`, keys are the level names when concatenating DataFrames.

```
display(s1)
display(s2)
display(s3)
```

```
a    0
b    1
dtype: int64
c    2
d    3
e    4
dtype: int64
f    5
g    6
dtype: int64
```

```
result = pd.concat([s1, s2, s3],
                    keys=['patient_1', 'patient_2', 'patient_3'])
```

```
result
```

```
patient_1 a    0
          b    1
patient_2 c    2
          d    3
          e    4
patient_3 f    5
          g    6
dtype: int64
```

```
result.unstack()
```

	a	b	c	d	e	f	g
patient_1	0.0	1.0	NaN	NaN	NaN	NaN	NaN
patient_2	NaN	NaN	2.0	3.0	4.0	NaN	NaN
patient_3	NaN	NaN	NaN	NaN	NaN	5.0	6.0

```
display(s1)
display(s2)
display(s3)
```

```
a    0
b    1
dtype: int64
c    2
d    3
e    4
dtype: int64
f    5
g    6
dtype: int64
```

```
pd.concat([s1, s2, s3], axis=1, keys=['COL_1', 'COL_2', 'COL_3'])
```

	COL_1	COL_2	COL_3
a	0.0	NaN	NaN
b	1.0	NaN	NaN
c	NaN	2.0	NaN
d	NaN	3.0	NaN
e	NaN	4.0	NaN
f	NaN	NaN	5.0
g	NaN	NaN	6.0

```
df1 = pd.DataFrame(np.arange(6).reshape(3, 2), index=['a', 'b', 'c'], columns=['one', 'two'])
df2 = pd.DataFrame(5 + np.arange(4).reshape(2, 2), index=['a', 'c'], columns=['three', 'four'])

display(df1)
display(df2)
```

	one	two
a	0	1
b	2	3
c	4	5

	three	four
a	5	6
c	7	8

```
pd.concat([df1, df2], axis=1, keys=['level1', 'level2'])
```

df1		
	one	two
a	0	1
b	2	3
c	4	5

	level1		level2	
	one	two	three	four
a	0	1	5.0	6.0
b	2	3	NaN	NaN
c	4	5	7.0	8.0

df2		
	three	four
a	5	6
c	7	8

```
pd.concat({'level1': df1, 'level2': df2}, axis=1)
```

	level1		level2	
	one	two	three	four
a	0	1	5.0	6.0
b	2	3	NaN	NaN
c	4	5	7.0	8.0

```
df1 = pd.DataFrame(np.random.randn(3, 4), columns=['a', 'b', 'c', 'd'])
df2 = pd.DataFrame(np.random.randn(2, 3), columns=['b', 'd', 'a'])

display(df1, df2)
```

	a	b	c	d
0	1.669025	-0.438570	-0.539741	0.476985
1	3.248944	-1.021228	-0.577087	0.124121
2	0.302614	0.523772	0.000940	1.343810

	b	d	a
0	-0.713544	-0.831154	-2.370232
1	-1.860761	-0.860757	0.560145

df1

	a	b	c	d
0	1.669025	-0.438570	-0.539741	0.476985
1	3.248944	-1.021228	-0.577087	0.124121
2	0.302614	0.523772	0.000940	1.343810

df2

	b	d	a
0	-0.713544	-0.831154	-2.370232
1	-1.860761	-0.860757	0.560145

```
pd.concat([df1, df2])
```

	a	b	c	d
0	1.669025	-0.438570	-0.539741	0.476985
1	3.248944	-1.021228	-0.577087	0.124121
2	0.302614	0.523772	0.000940	1.343810
0	-2.370232	-0.713544	NaN	-0.831154
1	0.560145	-1.860761	NaN	-0.860757

df1				
	a	b	c	d
0	1.669025	-0.438570	-0.539741	0.476985
1	3.248944	-1.021228	-0.577087	0.124121
2	0.302614	0.523772	0.000940	1.343810

pd.concat([df1, df2], ignore_index=True)

	a	b	c	d
0	1.669025	-0.438570	-0.539741	0.476985
1	3.248944	-1.021228	-0.577087	0.124121
2	0.302614	0.523772	0.000940	1.343810
3	-2.370232	-0.713544	NaN	-0.831154
4	0.560145	-1.860761	NaN	-0.860757

df2			
	b	d	a
0	-0.713544	-0.831154	-2.370232
1	-1.860761	-0.860757	0.560145

df1

	a	b	c	d
0	1.669025	-0.438570	-0.539741	0.476985
1	3.248944	-1.021228	-0.577087	0.124121
2	0.302614	0.523772	0.000940	1.343810

df2

	b	d	a
0	-0.713544	-0.831154	-2.370232
1	-1.860761	-0.860757	0.560145

```
pd.concat([df1, df2], axis=1)
```

	a	b	c	d	b	d	a
0	1.669025	-0.438570	-0.539741	0.476985	-0.713544	-0.831154	-2.370232
1	3.248944	-1.021228	-0.577087	0.124121	-1.860761	-0.860757	0.560145
2	0.302614	0.523772	0.000940	1.343810	NaN	NaN	NaN

df1									
	a	b	c	d					
0	1.669025	-0.438570	-0.539741	0.476985					
1	3.248944	-1.021228	-0.577087	0.124121					
2	0.302614	0.523772	0.000940	1.343810					

df2									
	b	d	a						
0	-0.713544	-0.831154	-2.370232						
1	-1.860761	-0.860757	0.560145						


```
pd.concat([df1, df2], join='inner')
```

	a	b	d	
0	1.669025	-0.438570	0.476985	
1	3.248944	-1.021228	0.124121	
2	0.302614	0.523772	1.343810	
0	-2.370232	-0.713544	-0.831154	
1	0.560145	-1.860761	-0.860757	

df1					df2				
	a	b	c	d		b	d	a	
0	1.669025	-0.438570	-0.539741	0.476985	0	-0.713544	-0.831154	-2.370232	
1	3.248944	-1.021228	-0.577087	0.124121	1	-1.860761	-0.860757	0.560145	
2	0.302614	0.523772	0.000940	1.343810					


```
pd.concat([df1, df2], axis=1, join='inner')
```

	a	b	c	d	b	d	a
0	1.669025	-0.438570	-0.539741	0.476985	-0.713544	-0.831154	-2.370232
1	3.248944	-1.021228	-0.577087	0.124121	-1.860761	-0.860757	0.560145

See the Table 8.3 in *Python for Data Analysis* for other useful `concat()` arguments.

Combining data with overlap

- `combine_first()` is a useful function for combining data using a method that is similar to numpy's `where()`.
 - Test some condition, if true take first value, else take second value.

```
s1 = pd.Series([np.nan, 2.5, np.nan, 3.5, 4.5, np.nan],
               index=['f', 'e', 'd', 'c', 'b', 'a'])
s2 = pd.Series(np.arange(len(s1), dtype=np.float64),
               index=['a', 'b', 'c', 'd', 'e', 'f'])
s2.iloc[-1] = np.nan

display(s1)
display(s2)
```

```
f    NaN
e    2.5
d    NaN
c    3.5
b    4.5
a    NaN
dtype: float64

a    0.0
b    1.0
c    2.0
d    3.0
e    4.0
f    NaN
dtype: float64
```

```
f    NaN
e    2.5
d    NaN
c    3.5
b    4.5
a    NaN
dtype: float64
a    0.0
b    1.0
c    2.0
d    3.0
e    4.0
f    NaN
dtype: float64
```

```
np.where(pd.isna(s1), s2, s1)
```

```
array([0. , 2.5, 2. , 3.5, 4.5, nan])
```

```
s1.combine_first(s2)
```

```
a    0.0
b    4.5
c    3.5
d    3.0
e    2.5
f    NaN
dtype: float64
```

```
df1 = pd.DataFrame({'a': [1., np.nan, 5., np.nan],
                    'b': [np.nan, 2., np.nan, 6.],
                    'c': range(2, 18, 4)})
df2 = pd.DataFrame({'a': [5., 4., np.nan, 3., 7.],
                    'b': [np.nan, 3., 4., 6., 8.]})

display(df1)
display(df2)
df1.combine_first(df2)
```

	a	b	c
0	1.0	NaN	2
1	NaN	2.0	6
2	5.0	NaN	10
3	NaN	6.0	14

	a	b
0	5.0	NaN
1	4.0	3.0
2	NaN	4.0
3	3.0	6.0
4	7.0	8.0

	a	b	c
0	1.0	NaN	2.0
1	4.0	2.0	6.0
2	5.0	4.0	10.0
3	3.0	6.0	14.0
4	7.0	8.0	NaN